Lab 10: The Priority Triage, Part A

CS 311: Data Structures and Algorithms (Fall 2026)

---

1. Array vs. Linked Tree, Index Formula Derivation

Deriving the formulas.

A complete binary tree fills one level at a time from left to right, and the array stores it in that order. Level d has 2^d nodes and starts at index 2^d - 1. Say node i is spot p on level d, so i = 2^d - 1 + p. Each node before it on that level has 2 children, so its left child is spot 2p on the next level.

```
left   = (2^(d+1) - 1) + 2p
       = 2(2^d - 1 + p) + 1
       = 2i + 1

right  = left + 1 = 2i + 2

parent: undo the child formulas
  i = 2j + 1  ->  j = (i - 1) / 2
  i = 2j + 2  ->  j = (i - 2) / 2
  both are the same as (i - 1) // 2
```

Sift-up from index 13.

```
13 -> parent (13 - 1) // 2 = 6
 6 -> parent ( 6 - 1) // 2 = 2
 2 -> parent ( 2 - 1) // 2 = 0   (root)
```

It compares against index 6, then 2, then 0.

---

2. Proof by Induction, Level Count

Base case (n = 1).

A heap with 1 node is just the root, so the height is 0. floor(log2(1)) = 0, so it works.

Inductive hypothesis.

Assume a complete binary tree with k nodes has height floor(log2(k)).

Inductive step (n = k + 1).

The new node goes in the next open spot on the bottom level.

Case 1: the bottom level was full. Then k = 2^h - 1, so the old height is floor(log2(2^h - 1)) = h - 1. The new node starts a new level, so the height becomes h, and floor(log2(2^h)) = h.

Case 2: the bottom level was not full. The new node fits on that level, so the height stays floor(log2(k)). Since k + 1 is not a power of 2, floor(log2(k + 1)) = floor(log2(k)).

Conclusion.

By induction a complete binary tree with n nodes has height floor(log2(n)).

---

3. Visual Tracing

Inserts.

```
insert 5: [5]
insert 3: [5, 3]        3 < 5 swap  -> [3, 5]
insert 8: [3, 5, 8]     8 > 3 no swap
insert 1: [3, 5, 8, 1]  1 < 5 swap  -> [3, 1, 8, 5]
                        1 < 3 swap  -> [1, 3, 8, 5]
insert 9: [1, 3, 8, 5, 9]     9 > 3 no swap
insert 2: [1, 3, 8, 5, 9, 2]  2 < 8 swap -> [1, 3, 2, 5, 9, 8]
                              2 > 1 no swap

final: [1, 3, 2, 5, 9, 8]
```

extract_min().

```
remove 1, move last (8) to the root: [8, 3, 2, 5, 9]
children 3 and 2, smaller is 2, 8 > 2 swap -> [2, 3, 8, 5, 9]
8 has no children, stop

returns 1, final: [2, 3, 8, 5, 9]
```

---

4. PDC Reasoning

Start with the heap [1, 3, 2, 5, 9, 8] and no locks.

```
1. Doctor calls extract_min and saves the root: 1
2. Intake A inserts 0 and sifts it all the way up:
   [0, 3, 1, 5, 9, 8, 2]
3. Doctor pops the last value (2) and puts it at the root:
   [2, 3, 1, 5, 9, 8]
4. Doctor sifts down and returns 1:
   [1, 3, 2, 5, 9, 8]
```

Patient 0 got written over in step 3, so that record is lost. Patient 1 was given to the doctor but is still in the heap, so it would get pulled again.